

COMPREHENSIVE TECHNICAL STUDY GUIDE

AudioSurvey AI

AI-POWERED MULTILINGUAL IVR SURVEY PLATFORM

Document Title **AudioSurvey AI — Complete Study Guide**

Version **v1.2.0**

Date April 6, 2026

Author Krishnanand

Status **PRODUCTION RELEASE**

Classification Internal — Presentation Ready

Technology Stack Python · Flask · Twilio · Azure TTS · Whisper · DeepFilterNet

ABSTRACT

AudioSurvey AI is a production-grade, AI-powered Interactive Voice Response (IVR) platform engineered to conduct automated multilingual telephone surveys targeting Kiswahili-speaking African refugee populations. The platform integrates Twilio telephony, Azure Neural TTS, OpenAI Whisper large-v3, DeepFilterNet noise suppression, and Google Translate

into a unified pipeline capable of placing outbound calls, receiving inbound calls, conducting 76-question surveys, and processing responses through a 4-stage ML pipeline — all managed through a secure web-based admin dashboard.

CONFIDENTIAL — FOR INTERNAL PRESENTATION USE ONLY

© 2026 AudioSurvey AI Project · All Rights Reserved

AudioSurvey AI — Complete Study Guide for Presentation

Version 1.2.0 | April 2026 | Comprehensive Code-Level Walkthrough

TABLE OF CONTENTS

1. [Project Overview](#)
2. [Version History](#)
3. [Technology Stack](#)
4. [Project Structure — Every File Explained](#)
5. [How the Application Starts](#)
6. [Core Application — twilio_handler.py](#)
7. [State Management — state.py](#)
8. [Call Scheduler — scheduler.py](#)
9. [Background ML Worker — background_worker.py](#)
10. [Audio Preprocessing — audio_preprocess.py](#)
11. [Whisper Transcription — transcribe.py](#)
12. [Translation — translate.py](#)
13. [Text-to-Speech — tts.py](#)
14. [Excel Export — export_excel.py](#)
15. [Survey Question Format — questions.txt](#)
16. [Runtime Status Module — runtime_status.py](#)
17. [Inbound Call Handling](#)
18. [Complete End-to-End Data Flow](#)
19. [Thread Architecture and Concurrency](#)
20. [Configuration Files](#)
21. [Security Features](#)
22. [Key Design Decisions and Why](#)
23. [Libraries and What Each One Does](#)

24. [Glossary](#)

25. [Presentation Talking Points](#)

1. PROJECT OVERVIEW

What Is AudioSurvey AI?

AudioSurvey AI is an **AI-powered multilingual Interactive Voice Response (IVR) survey platform** that conducts automated telephone-based research surveys. It targets **African refugees who speak Kiswahili**, enabling researchers to collect structured survey responses at scale without in-person enumerators.

Core Idea

Instead of sending a person to interview participants face-to-face, the system **places automated phone calls** (and accepts inbound calls from participants), asks questions in **Kiswahili using AI-generated voice**, collects responses (voice or keypad presses), and then uses an **AI/ML pipeline** to process and translate everything to English.

Key Capabilities

Feature	How It Works
Automated Outbound Calling	Scheduler dials participants via Twilio at scheduled times
Inbound Call Support	Participants can call in and complete the survey (v1.2.0)
Multilingual IVR Prompts	Azure Neural TTS speaks questions in Kiswahili
4 Question Types	INFO (instruction), OPEN (speech), MCQ (keypad), MCQO (keypad + "Other" speech)
Full Call Recording	Entire call recorded (up to 30 min) via Twilio
AI Noise Removal	DeepFilterNet (PyTorch) cleans background noise
AI Transcription	OpenAI Whisper large-v3 converts Kiswahili speech to text
Machine Translation	Google Translate converts Kiswahili text to English
English Audio Generation	Google TTS creates English audio from translations
Excel Data Export	Structured MCQ/MCQO responses exported to spreadsheets
Admin Dashboard	Web UI for managing participants, scheduling, monitoring
Conference Calling	Three-way calls for researcher-moderated interviews
Runtime Health Monitoring	Heartbeat-based status for scheduler + worker threads (v1.2.0)
Call Direction Tracking	Records whether each call was inbound or outbound (v1.2.0)
End Call Controls	Admin can end individual or all active calls (v1.2.0)

2. VERSION HISTORY

Version	Date	Author	Summary
v0.1.0	Jan 22, 2026	Krishnanand	Initial commit — base Flask app, Twilio IVR skeleton
v0.2.0	Jan 22, 2026	Krishnanand	Added scheduler, background worker, state management
v0.3.0	Jan 22, 2026	Krishnanand	Azure TTS integration and IVR audio caching
v0.4.0	Jan 22, 2026	Krishnanand	Authentication system with brute-force protection
v0.5.0	Jan 22, 2026	Krishnanand	DeepFilterNet audio preprocessing pipeline
v0.6.0	Jan 22, 2026	Krishnanand	Whisper transcription and Google Translate integration
v0.7.0	Jan 22, 2026	Krishnanand	Excel export with MCQ/MCQO digit decoding
v0.8.0	Jan 22, 2026	Krishnanand	Admin dashboard (Flask Blueprint)
v0.9.0	Jan 22, 2026	Krishnanand	Conference call (3-way calling) feature
v0.10.0	Jan 22, 2026	Krishnanand	ngrok auto-launch via run_app.py
v1.0.0	Jan 22, 2026	Krishnanand	First stable release — full pipeline operational
v1.0.1	Jan 22, 2026	Krishnanand	Call log CSV audit trail
v1.0.2	Feb 10, 2026	Krishnanand	Scheduler force-call and pause/resume controls
v1.0.3	Feb 10, 2026	Krishnanand	Admin token bypass for API access
v1.0.4	Feb 22, 2026	Krishnanand	MCQO "Other" speech follow-up handler
v1.0.5	Mar 12, 2026	Krishnanand	runtime_warnings.py to suppress noisy library output
v1.0.6	Mar 17, 2026	Krishnanand	Changed file naming convention for recordings
v1.1.0	Mar 21, 2026	Krishnanand	Full audio recording logic overhaul (Start+Recording TwiML)
v1.1.1	Mar 21, 2026	Krishnanand	Dashboard improvements and participant CSV import
v1.2.0-rc1	Apr 5, 2026	Krishnanand	Inbound call handling, call direction tracking
v1.2.0-rc2	Apr 5, 2026	Krishnanand	End Call / End All Calls admin controls
v1.2.0	Apr 6, 2026	Krishnanand	Production release: runtime_status.py, admin dashboard v2

3. TECHNOLOGY STACK

Programming Language

- **Python 3.x** — the entire application is written in Python

Web Framework

- **Flask 3.1.2** — lightweight web framework that handles all HTTP endpoints (webhooks from Twilio, admin dashboard, API routes)

Telephony (Phone Calls)

- **Twilio Voice API** — cloud service that places and receives phone calls
- **TwimL (Twilio Markup Language)** — XML-based language to script what happens during a call (play audio, gather input, record, hangup)

AI/ML Models

- **OpenAI Whisper (large-v3)** — state-of-the-art speech recognition model. Transcribes Kiswahili audio to text. Runs locally on the machine (not cloud API)
- **DeepFilterNet** — PyTorch-based deep learning model for removing background noise from audio recordings
- **Azure Cognitive Services (Neural TTS)** — Microsoft's cloud service for generating natural-sounding Kiswahili voice prompts using SSML
- **Google TTS (gTTS)** — Google's text-to-speech for generating English audio output
- **Google Translate (googletrans)** — translates Kiswahili text to English via scraping

Audio Processing

- **FFmpeg** — command-line tool for audio format conversion and resampling
- **PyTorch + torchaudio** — ML framework powering DeepFilterNet
- **NumPy** — numerical computing for audio data manipulation
- **pydub** — Python audio manipulation library

Data and Export

- **pandas** — data manipulation and analysis
- **openpyxl** — Excel file (.xlsx) generation
- **PyYAML** — YAML configuration file parsing

Infrastructure

- **ngrok** — creates secure HTTPS tunnels so Twilio can reach the local Flask server
- **gunicorn** — production WSGI server

Security

- **werkzeug** — PBKDF2 password hashing for admin authentication
 - **python-dotenv** — loads API keys from `.env` file
-

4. PROJECT STRUCTURE — EVERY FILE EXPLAINED


```

audiosurvey_ai/
|
|-- run_app.py           # ENTRY POINT: Starts ngrok + Flask server + opens browser
|-- main.py             # Standalone batch processor (transcribe/translate/TTS offline)
|-- config.yaml          # All settings: Twilio, IVR, audio processing, auth
|-- .env                # Secret API keys (Twilio, Azure, etc.)
|-- requirements.txt     # 146 Python package dependencies
|
|-- app/                # Main application code
|   |-- __init__.py      # Makes 'app' a Python package
|   |-- twilio_handler.py # CORE: Flask app + all webhook routes (~1,400 lines)
|   |-- dashboard.py     # Admin web UI (Flask Blueprint)
|   |-- state.py         # Participant state management (JSON persistence)
|   |-- background_worker.py # ML processing pipeline (continuous worker thread)
|   |-- scheduler.py     # Automated call scheduler (background thread)
|   |-- runtime_status.py # Heartbeat health monitor for background threads (v1.2.0)
|   |-- audio_preprocess.py # DeepFilterNet noise removal pipeline
|   |-- transcribe.py    # OpenAI Whisper speech-to-text
|   |-- translate.py     # Google Translate with chunking + retry
|   |-- tts.py           # Google TTS (English audio output)
|   |-- azure_tts.py     # Azure Neural TTS (IVR Kiswahili prompts)
|   |-- export_excel.py  # Survey response export to Excel
|   |-- utils.py         # Scheduling helper (time conversion)
|   |-- twilio_utils.py  # Twilio call helpers
|   |-- file_naming.py   # Safe filename generation
|   |-- logger.py        # Colored console logging
|   |-- auth.py          # Authentication module
|   |-- runtime_warnings.py # Suppresses noisy library warnings
|
|-- data/                # All data storage (file-based, no database)
|   |-- state/
|   |   |-- participants.json # Main state file: all participant data + responses
|   |   |-- call_log.csv      # Audit log of every call event
|   |   |-- settings.json     # Global settings (paused flag)
|   |-- questions.txt        # Survey questions (76 questions, pipe-delimited format)
|   |-- contacts.csv         # Phone contact list template
|   |-- audio/               # Raw call recordings (.wav)
|   |-- audio_processed/     # Cleaned audio after DeepFilterNet (.wav)
|   |-- transcripts/         # Whisper transcriptions (.txt, Kiswahili)
|   |-- translations/        # Google Translate output (.txt, English)
|   |-- english_audio/       # gTTS English audio (.mp3)
|   |-- ivr_audio/           # Cached Azure TTS prompts (.mp3)
|   |-- results/             # Excel exports (.xlsx)
|
|-- packaging/
|   |-- macos_dmg/           # macOS .app bundle and DMG installer

```

5. HOW THE APPLICATION STARTS

Mode 1: Normal Launch — run_app.py

When you run `python3 run_app.py`, here is exactly what happens:

```
# Step 1: Start ngrok tunnel
ngrok_proc = subprocess.Popen(["ngrok", "http", "5050"])
# ngrok creates an HTTPS URL like: https://armlike-unadduced-ai.ngrok-free.dev
# This is needed because Twilio requires HTTPS to send webhooks

# Step 2: Fetch the tunnel URL
# Queries ngrok's local API at http://127.0.0.1:4040/api/tunnels
# Extracts the HTTPS public URL

# Step 3: Set environment variable
os.environ["PUBLIC_BASE_URL"] = "https://armlike-unadduced-ai.ngrok-free.dev"

# Step 4: Launch Flask server as subprocess
subprocess.Popen([sys.executable, "-m", "app.twilio_handler", "serve"])

# Step 5: Open browser to admin dashboard
webbrowser.open("http://127.0.0.1:5050/admin")
```

Mode 2: Direct Server — python3 -m app.twilio_handler serve

```
# Step 1: Start background services
start_background_services()
#   -> Starts Scheduler thread (daemon, polls every 15 seconds)
#   -> Starts ML Worker thread (daemon, polls every 5 seconds)

# Step 2: Run Flask web server
app.run(host="0.0.0.0", port=5050, debug=False, use_reloader=False)
```

Mode 3: Batch Processing — main.py

```
# Processes audio files offline (no calls needed)
# 1. Transcribe all audio files in data/audio/ using Whisper
# 2. Translate all transcripts from Kiswahili to English
# 3. Generate English audio from translations
```

6. CORE APPLICATION — twilio_handler.py

This is the biggest and most important file (~1,400 lines). Here is every section:

6.1 Imports and Configuration

```

from dotenv import load_dotenv
load_dotenv()

TWILIO_SID = os.getenv("TWILIO_ACCOUNT_SID")
TWILIO_TOKEN = os.getenv("TWILIO_AUTH_TOKEN")
TWILIO_FROM = os.getenv("TWILIO_FROM_NUMBER")
PUBLIC_BASE_URL = os.getenv("PUBLIC_BASE_URL")

GATHER_TIMEOUT = 6 # Wait 6 seconds for speech/keypad input
SPEECH_TIMEOUT = "auto" # Auto-detect end of speech
SPEECH_LANGUAGE = "sw-KE" # Kiswahili (Kenya)
RECORDING_MAX_SEC = 1800 # 30 minutes max

```

6.2 Azure TTS for IVR Prompts

```

AZURE_TTS_VOICE_SW = "sw-KE-ZuriNeural" # Kiswahili voice
AZURE_TTS_VOICE_EN = "en-US-JennyNeural" # English voice

def get_prompt_audio_url(text, lang):
    """Returns a PUBLIC URL to a cached MP3 of the prompt"""

    voice = AZURE_TTS_VOICE_SW if lang.startswith("sw") else AZURE_TTS_VOICE_EN

    # Cache key: SHA1 hash of voice+text – same text is never synthesized twice
    key = hashlib.sha1(f"{voice}|mp3|{text}".encode("utf-8")).hexdigest()
    filename = f"{key}.mp3"
    out_path = os.path.join(IVR_AUDIO_DIR, filename)

    if not os.path.exists(out_path) or os.path.getsize(out_path) < 2000:
        # Build SSML with -15% prosody rate for clearer Kiswahili speech
        ssml = f"""
        <speak version="1.0" xml:lang="sw-KE">
            <voice name="{voice}">
                <prosody rate="-15%">{text}</prosody>
            </voice>
        </speak>"""
        synthesizer.speak_ssml_async(ssml).get()

    return f"{PUBLIC_BASE_URL}/ivr-audio/{filename}"

```

Key concept: TTS audio is **cached by text hash**. Same text = same file, so Azure is never called twice for the same question.

6.3 Authentication System

```

AUTH_MAX_FAILS      = 7      # Max failed attempts before logout
AUTH_LOCK_SECONDS   = 900    # 15-minute logout
AUTH_WINDOW_SECONDS = 600    # 10-minute window for counting failures

@app.route("/login", methods=["GET", "POST"])
def login_route():
    # 1. Check if account is locked -> "Try again in X seconds"
    # 2. Verify PBKDF2 password hash
    # 3. On failure: record failure, increment counter
    # 4. On success: clear failures, create 8-hour session

```

6.4 Flask App Setup

```

app = Flask(__name__)
app.register_blueprint/dashboard_bp) # Admin dashboard routes

app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
    SESSION_COOKIE_SECURE=True,
    PERMANENT_SESSION_LIFETIME=timedelta(hours=8),
)

@app.before_request
def _guard_admin_routes():
    # /voice, /start, /next etc. are PUBLIC (Twilio webhooks)
    # /admin/* requires valid session or ADMIN_TOKEN header
    if path.startswith("/admin"):
        if not _is_logged_in():
            return redirect("/login")

```

6.5 IVR Call Flow Routes

Route: /voice — Call Starts Here

```

@app.route("/voice", methods=["POST"])
def voice():
    """Called by Twilio the moment participant answers"""
    xml = """
    <Response>
      <Start>
        <Recording maxLength="1800"
          recordingStatusCallback="/recording-done"
          recordingStatusCallbackEvent="completed" />
      </Start>
      <Redirect>/start</Redirect>
    </Response>"""

```

Route: /start — Survey Begins

```
@app.route("/start", methods=["POST"])
def start():
    # Play 2 INFO questions (intro)
    # Then play first real question with <Gather>
    xml = f"""
    <Response>
        <Play>{intro1_url}</Play>
        <Play>{intro2_url}</Play>
        <Gather input="speech" timeout="6" speechTimeout="auto" action="/next?q=3">
            <Play>{q3_url}</Play>
        </Gather>
        <Redirect>/next?q=3</Redirect>
    </Response>"""
```

Route: /next — Main Question Loop

```

@app.route("/next", methods=["POST", "GET"])
def next_question():
    q = int(request.args.get("q", "0"))
    speech = request.values.get("SpeechResult", "").strip()

    # Store previous speech answer
    if looks_like_real_speech(speech):
        state[pid]["responses"][f"q{counter}"] = speech

    # Survey done?
    if q >= len(questions):
        return twiml("<Response><Play>kwaheri.mp3</Play><Hangup/></Response>")

    if question["type"] in ["mcq", "mcqo"]:
        return twiml(f"""
        <Response>
            <Gather input="dtmf" numDigits="1" timeout="6" action="/mcq-handler?q={q}">
                <Play>{q_url}</Play>
            </Gather>
            <Redirect>/next?q={q}</Redirect>
        </Response>""")

    elif question["type"] == "info":
        return twiml(f"""
        <Response>
            <Play>{q_url}</Play>
            <Redirect>/next?q={q+1}</Redirect>
        </Response>""")

    else: # OPEN
        return twiml(f"""
        <Response>
            <Gather input="speech" timeout="6" speechTimeout="auto" action="/next?q={q+1}">
                <Play>{q_url}</Play>
            </Gather>
            <Redirect>/next?q={q+1}</Redirect>
        </Response>""")

```

Route: /mcq-handler — Keypad Input

```
@app.route("/mcq-handler", methods=["POST"])
def mcq_handler():
    digit = request.form.get("Digits", "") # "1", "2", or "3"
    state[pid]["responses"][f"q{counter}"] = digit

    # MCQ0: if "Other" digit selected, collect speech
    if question["type"] == "mcq" and digit == other_digit:
        return twiml(f"""
        <Response>
            <Play>{please_speak_url}</Play>
            <Gather input="speech" timeout="4" action="/mcq-other-handler?q={q}"/>
            <Redirect>/next?q={q+1}</Redirect>
        </Response>""")

    return twiml(f"<Response><Redirect>/next?q={q+1}</Redirect></Response>")
```

Route: /recording-done — Recording Callback

```
@app.route("/recording-done", methods=["POST"])
def recording_done():
    # Download WAV from Twilio
    r = requests.get(recording_url + ".wav", auth=(TWILIO_SID, TWILIO_TOKEN))
    with open(audio_path, "wb") as f:
        f.write(r.content)

    # Queue for ML processing if participant engaged
    state[pid]["processing_status"] = "pending" if engaged else "saved_no_engagement"
```

Route: /admin/end_call — End Individual Call (v1.2.0)

```
@app.route("/admin/end_call", methods=["POST"])
def admin_end_call():
    call_sid = request.form.get("call_sid")
    client.calls(call_sid).update(status="completed")
    return jsonify({"ok": True})
```

Route: /admin/end_all_calls — End All Calls (v1.2.0)

```
@app.route("/admin/end_all_calls", methods=["POST"])
def admin_end_all_calls():
    for pid, p in state.items():
        if p.get("last_call_status") in {"in-progress", "ringing"}:
            client.calls(p["last_call_sid"]).update(status="completed")
    return jsonify({"ok": True, "terminated": terminated})
```

7. STATE MANAGEMENT — state.py

No Database — Everything in JSON Files

```
PARTICIPANTS_PATH = "data/state/participants.json"
CALL_LOG_PATH      = "data/state/call_log.csv"
SETTINGS_PATH      = "data/state/settings.json"
STATE_IO_LOCK      = threading.RLock() # Reentrant lock for thread safety
```

Participant Data Model (v1.2.0)

```
{
  "participant_001": {
    "phone_e164": "+1234567890",
    "status": "completed",
    "attempts": 2,
    "last_call_time": "2026-03-21T14:30:00",
    "last_call_sid": "CA123abc...",
    "last_call_status": "completed",
    "direction": "outbound-api",
    "engaged": true,
    "processing_status": "completed",
    "scheduled_time_utc": "2026-03-21T18:30:00Z",
    "responses": {
      "q1": "Jina langu ni Amina",
      "q2": "2",
      "q3": "1",
      "survey_q_counter": 15
    },
    "last_outputs": {
      "audio_path": "data/audio/participant_001_20260321_143000.wav",
      "processed_audio_path": "data/audio_processed/...",
      "transcript_path": "data/transcripts/...",
      "translation_path": "data/translations/...",
      "english_audio_path": "data/english_audio/..."
    }
  }
}
```

New in v1.2.0: The `direction` field records `"inbound"` or `"outbound-api"` for every call.

Atomic File Write


```
def save_participants(state):  
    with STATE_IO_LOCK:  
        tmp_path = f"{PARTICIPANTS_PATH}.{threading.get_ident()}.tmp"  
        with open(tmp_path, "w", encoding="utf-8") as f:  
            json.dump(state, f, ensure_ascii=False, indent=2)  
        os.replace(tmp_path, PARTICIPANTS_PATH) # Atomic – no corrupt files on crash
```

Call Eligibility Logic

```
def can_call(state, participant_id, force=False):  
    p = state.get(participant_id)  
  
    if p["status"] in {"completed", "failed"}: return False # Never retry  
    if p["attempts"] >= 3: return False # Max attempts  
    if force: return True # Skip schedule  
    if datetime.now() < scheduled_time: return False # Not yet  
    if (now - last_call_time) < timedelta(hours=1): return False # Too soon  
  
    return True
```

8. CALL SCHEDULER — scheduler.py

```
def run_once(force=False):
    if is_paused() and not force:
        return # Admin paused calls

    client = Client(TWILIO_SID, TWILIO_TOKEN)
    state = load_participants()

    for participant_id, p in state.items():
        if not can_call(state, participant_id, force=force):
            continue

        call = client.calls.create(
            to=p["phone_e164"],
            from_=TWILIO_FROM,
            url=f"{PUBLIC_BASE_URL}/voice",
            record=True,
            recording_status_callback=f"{PUBLIC_BASE_URL}/recording-done",
            status_callback=f"{PUBLIC_BASE_URL}/call-status",
            status_callback_event=["completed", "no-answer", "busy", "failed", "canceled"],
        )
        mark_call_started(state, participant_id, call.sid, direction="outgoing")

def start_scheduler_in_background(interval_sec=15):
    def _loop():
        mark_scheduler_started() # Notify runtime_status
        time.sleep(interval_sec) # Wait before first tick
        while True:
            mark_scheduler_heartbeat()
            run_once(force=False)
            mark_scheduler_heartbeat()
            time.sleep(interval_sec)

    threading.Thread(target=_loop, daemon=True).start()
```

9. BACKGROUND ML WORKER — background_worker.py

```
def process_pending_recordings():
    while True:
        mark_worker_heartbeat() # Signal health to runtime_status
        state = load_participants()

        for pid, p in state.items():
            if p.get("processing_status") != "pending":
                continue

            state[pid]["processing_status"] = "processing"
            save_participants(state)

            # Stage 1: DeepFilterNet noise removal
            processed_path = preprocess_recording(audio_path)

            # Stage 2: Whisper large-v3 transcription
            text, detected_lang = transcribe_audio(processed_path)

            # Stage 3: Google Translate (Kiswahili -> English)
            english_text = text if detected_lang == "en" else translate_to_english_chunked(text)

            # Stage 4: Google TTS -> English MP3
            text_to_english_audio(english_text, english_audio_path)

            state[pid]["processing_status"] = "completed"
            save_participants(state)

        time.sleep(5) # Poll every 5 seconds
```

10. AUDIO PREPROCESSING — audio_preprocess.py

Three-Step Pipeline

Raw WAV -> FFmpeg (48kHz mono) -> DeepFilterNet (denoise) -> FFmpeg (16kHz for Whisper)

Step 1: Prepare for Model

```
# Convert to 48kHz mono PCM (DeepFilterNet was trained on 48kHz)
cmd = ["ffmpeg", "-y", "-i", src_path, "-ac", "1", "-ar", "48000", "-c:a", "pcm_s16le", prep_p
```

Step 2: DeepFilterNet Noise Removal

```
# Read WAV -> numpy array -> PyTorch tensor
arr = np.frombuffer(frames, dtype=np.int16).astype(np.float32) / 32768.0
audio_tensor = torch.from_numpy(arr).unsqueeze(0) # Add batch dimension

# Run deep learning model
enhanced = enhance_fn(model, df_state, audio_tensor)

# Write back to WAV (scale [-1.0, 1.0] -> int16)
pcm = (enhanced.detach().cpu().squeeze(0).clamp(-1.0, 1.0).numpy() * 32767.0).astype(np.int16)
```

Step 3: Resample for Whisper

```
# Whisper needs 16kHz input
cmd = ["ffmpeg", "-y", "-i", enhanced_path, "-ac", "1", "-ar", "16000", final_path]
```

11. WHISPER TRANSCRIPTION — transcribe.py

```
model = whisper.load_model("large-v3") # Loaded once at startup

def transcribe_audio(file_path):
    result = model.transcribe(
        file_path,
        language="sw", # Force Kiswahili
        task="transcribe",
        fp16=False, # CPU safe (float32)
        condition_on_previous_text=False, # Prevents hallucination drift
        temperature=0.0 # Greedy decoding - most confident word
    )
    return result["text"], result.get("language")
```

Key Parameters: - `language="sw"` — Forces Kiswahili; without this, Whisper might misdetect the language - `condition_on_previous_text=False` — Prevents repeating phrases when audio is unclear - `temperature=0.0` — Always picks the most likely word (deterministic output)

12. TRANSLATION — translate.py

```
translator = Translator()
MAX_CHARS = 3000 # Google Translate limit per request

def translate_to_english_chunked(text, retries=3, sleep_sec=1.5):
    chunks = _split_text(text) # Split at sentence boundaries
    out_chunks = []

    for idx, chunk in enumerate(chunks):
        for attempt in range(retries):
            try:
                res = translator.translate(chunk, src="sw", dest="en")
                out_chunks.append(res.text)
                break
            except Exception:
                time.sleep(sleep_sec)

        if len(out_chunks) < idx + 1:
            out_chunks.append(f"[TRANSLATION_FAILED_CHUNK {idx+1}]")

    return "\n".join(out_chunks)
```

Why chunking? Google Translate has a ~5000 character limit. Using 3000-char chunks with 3 retries prevents failures on long transcripts.

13. TEXT-TO-SPEECH — tts.py

```
from gtts import gTTS

def text_to_english_audio(text, out_path):
    text = text.strip()
    if not text or "TRANSLATION_FAILED" in text:
        return False

    tts = gTTS(text=text, lang="en")
    tts.save(out_path) # Saves as MP3
    return True
```

14. EXCEL EXPORT — export_excel.py

```
def build_export_rows(state):
    metadata = _build_response_metadata()
    # {"q1": {"type": "open"}, "q2": {"type": "mcq", "options": ["Opt A", "Opt B", ...]}}

    for pid, pdata in state.items():
        filtered = filter_responses_for_excel(responses, metadata)

        row = {"participant_id": pid}
        for key, value in filtered.items():
            # Decode stored digit to option text
            # "2" -> options[1] -> "Kumaliza masomo"
            row[export_key] = _decode_choice_value(value, rule["options"])

        rows.append(row)

def _decode_choice_value(value, options):
    digit = int(float(value)) # "2" -> 2
    return options[digit - 1] # options[1] -> "Kumaliza masomo"
```

15. SURVEY QUESTION FORMAT — questions.txt

Format

Each line: `TYPE|Question Text|Option1|Option2|Option3...`

Type	Format	Response	Example
INFO	INFO\ Text	None (instruction)	INFO\ Karibu kwenye utafiti wetu...
OPEN	OPEN\ Text	Speech (6 sec)	OPEN\ Tafadhali sema jina lako
MCQ	MCQ\ Text\ Opt1\ Opt2\ Opt3	1 keypad digit	MCQ\ Mado anataka nini? \ Kuhamia\ Kumaliza\ Kufungua
MCQO	MCQO\ Text\ Opt1\ Opt2\ Nyingine	Digit + speech if "Other"	MCQO\ Kupanga watoto...\ Ndiyo\ Hapana\ Nyingine

Question Flow in a Call

```

questions[0] = INFO  -> Play intro (no response)
questions[1] = INFO  -> Play intro (no response)
questions[2] = OPEN  -> First real question (speech)
questions[3] = MCQ   -> Keypad press
questions[4] = MCQ0  -> Keypad press (+ speech if "Other")
...
questions[75] = MCQ   -> Last question
                  -> "Kwaheri" (Goodbye) -> Hangup

```

16. RUNTIME STATUS MODULE — runtime_status.py (v1.2.0)

Purpose

Provides **heartbeat-based health monitoring** for the scheduler and ML worker threads. The admin dashboard polls this to show live service status.

How It Works

Both threads call heartbeat functions at regular intervals. If a thread stops calling its heartbeat (crashed or hung), the monitor detects it as "down" once the heartbeat age exceeds the timeout.

```

SCHEDULER_TIMEOUT_SEC = 45  # "down" if no heartbeat for 45 sec
WORKER_TIMEOUT_SEC    = 20  # "down" if no heartbeat for 20 sec
STATUS_LOCK = threading.RLock()

# Internal timestamps (None until thread starts)
_scheduled_started_at: Optional[float] = None
_scheduled_heartbeat_at: Optional[float] = None
_worker_started_at: Optional[float] = None
_worker_heartbeat_at: Optional[float] = None

```

Heartbeat Functions

```

def mark_scheduler_started():
    with STATUS_LOCK:
        _scheduled_started_at = _scheduled_started_at or time.time()
        _scheduled_heartbeat_at = time.time()

def mark_scheduler_heartbeat():
    with STATUS_LOCK:
        _scheduled_heartbeat_at = time.time()

# Same pattern for mark_worker_started() and mark_worker_heartbeat()

```

Status Logic

```
def _service_snapshot(started_at, heartbeat_at, timeout_sec, paused=False):
    now = time.time()
    age_sec = None if heartbeat_at is None else int(now - heartbeat_at)

    if not started_at:
        return {"status": "not_started", "label": "Not started", "age_sec": age_sec}
    if heartbeat_at is None:
        return {"status": "starting", "label": "Starting", "age_sec": age_sec}
    if (now - heartbeat_at) > timeout_sec:
        return {"status": "down", "label": "Down", "age_sec": age_sec}
    if paused:
        return {"status": "paused", "label": "Paused", "age_sec": age_sec}
    return {"status": "running", "label": "Running", "age_sec": age_sec}
```

Status Values

Status	Meaning
<code>not_started</code>	Thread has never been started
<code>starting</code>	Thread started but no heartbeat received yet
<code>running</code>	Heartbeat received within the timeout window
<code>paused</code>	Scheduler: admin has paused outbound calls
<code>down</code>	No heartbeat within timeout — thread likely crashed

Dashboard Usage

```
# GET /admin/runtime_status
snapshot = get_runtime_snapshot(paused=is_paused())
# Returns:
# {
#   "scheduler": {"status": "running", "label": "Running", "age_sec": 3},
#   "worker":    {"status": "running", "label": "Running", "age_sec": 1}
# }
```

17. INBOUND CALL HANDLING (v1.2.0)

What Is an Inbound Call?

Participants can **call in to the Twilio number** instead of waiting to be called. Twilio sends the same `/voice` webhook, but with `Direction=inbound`. The system matches the caller to an existing participant or auto-creates a new one.

Direction Detection

```
@app.route("/voice", methods=["POST"])
def voice():
    direction = request.values.get("Direction", "outbound-api")
    caller_phone = request.values.get("From", "").strip()
    call_sid = request.values.get("CallSid", "")

    if direction == "inbound":
        state = load_participants()
        pid = _find_pid_by_phone(state, caller_phone)

        if pid is None:
            pid = _create_inbound_participant(state, caller_phone, call_sid)
        else:
            mark_call_started(state, pid, call_sid, direction="inbound")
            save_participants(state)
    # ... continue to recording + survey
```

Finding Participant by Phone

```
def _find_pid_by_phone(state, phone_e164):
    for pid, p in state.items():
        if p.get("phone_e164") == phone_e164:
            return pid
    return None # Unknown caller -> will auto-create
```

Auto-Creating New Inbound Participants

```
def _create_inbound_participant(state, phone_e164, call_sid):
    pid = f"inbound_{int(time.time())}_{phone_e164[-4:]}"
    state[pid] = {
        "phone_e164": phone_e164,
        "status": "in_progress",
        "attempts": 1,
        "last_call_sid": call_sid,
        "last_call_time": datetime.utcnow().isoformat(),
        "direction": "inbound",
        "engaged": False,
        "processing_status": "not_started",
        "responses": {},
        "survey_q_counter": 0,
    }
    save_participants(state)
    return pid
```

Direction in State

`mark_call_started()` records the direction on every call:

```
def mark_call_started(state, participant_id, call_sid, direction="outbound-api"):
    p = state[participant_id]
    p["last_call_sid"] = call_sid
    p["last_call_time"] = datetime.utcnow().isoformat()
    p["last_call_status"] = "in-progress"
    p["direction"] = direction # "inbound" or "outbound-api"
    p["attempts"] = p.get("attempts", 0) + 1
    p["status"] = "in_progress"
```

The admin dashboard **Direction column** shows `inbound` or `outbound-api` for each participant, letting researchers distinguish self-initiated calls from scheduled calls.

18. COMPLETE END-TO-END DATA FLOW

PHASE 1: SCHEDULING

Admin uploads CSV with phone numbers via dashboard
 Admin sets scheduled time for each participant
 Participants stored in data/state/participants.json

PHASE 2: CALLING

OUTBOUND: Scheduler checks every 15 seconds
 -> can_call() true -> Twilio places call -> /voice (Direction=outbound-api)
 INBOUND: Participant dials the Twilio number
 -> Twilio sends /voice webhook (Direction=inbound)
 -> System matches phone or auto-creates participant

PHASE 3: IVR SURVEY

/voice: Detect direction, match/create participant, start recording, redirect /start
 /start: Play 2 INFO intros, first OPEN question
 /next: Loop through all 76 questions
 OPEN: <Gather speech> -> store Kiswahili text
 MCQ: <Gather dtmf> -> store digit ("1", "2", "3")
 MCQ0: <Gather dtmf> -> digit + optional speech if "Other"
 INFO: <Play> -> auto-advance
 Done: "Kwaheri" -> <Hangup>

PHASE 4: RECORDING DOWNLOAD

Twilio calls /recording-done callback
 Download full WAV from Twilio API
 Save to data/audio/participant_001_20260321_143000.wav
 Set processing_status = "pending"

PHASE 5: ML PROCESSING (Background Worker, 5-second poll)

1. DeepFilterNet: noisy WAV -> 48kHz denoise -> 16kHz clean WAV
2. Whisper large-v3: 16kHz WAV -> Kiswahili text
3. Google Translate: Kiswahili -> English text
4. gTTS: English text -> English MP3

PHASE 6: DATA EXPORT

Admin clicks Export Excel
 MCQ/MCQ0 digits decoded to option text
 Output: data/results/ivr_responses.xlsx
 Optional: English translation -> ivr_responses_english.xlsx

19. THREAD ARCHITECTURE AND CONCURRENCY

```

Main Process (Python)
|
|-- Main Thread (Flask WSGI Server, port 5050)
|   Handles all HTTP: /voice, /start, /next, /admin/*
|
|-- Scheduler Thread (daemon, 15-second interval)
|   mark_scheduler_heartbeat() on every tick
|   can_call() check -> Twilio calls.create()
|
|-- ML Worker Thread (daemon, 5-second interval)
|   mark_worker_heartbeat() on every iteration
|   denoise -> transcribe -> translate -> TTS
|
All threads share STATE_IO_LOCK (RLock)
    -> Prevents concurrent writes to participants.json
    -> Atomic writes via temp file + os.replace()

```

Why daemon threads? They die automatically when the main process exits — no cleanup needed.

Why RLock (not Lock)? RLock allows the same thread to re-acquire the lock it already holds. Prevents deadlock when `save_participants()` is called from code that already holds the lock.

Why atomic writes? Writing to a temp file and then calling `os.replace()` is atomic at the OS level. If the app crashes mid-write, the original `participants.json` is intact.

20. CONFIGURATION FILES

config.yaml

```
twilio:
  account_sid: ""          # Overridden by .env
  auth_token: ""
  from_number: ""

ivr:
  questions_file: "data/questions.txt"
  gather_timeout_sec: 6
  speech_timeout: "auto"
  speech_language: "sw-KE"
  recording_max_seconds: 1800

audio_processing:
  enabled: true
  backend: "deepfilternet"
  model_sample_rate: 48000 # DeepFilterNet input
  output_sample_rate: 16000 # Whisper input
  channel_mode: "mixdown"

auth:
  users:
    krishnanand:
      password_hash: "pbkdf2:sha256:1000000$..."
```

.env (Secret Keys)

```
TWILIO_ACCOUNT_SID=ACf2d28a...
TWILIO_AUTH_TOKEN=21a1b082...
TWILIO_FROM_NUMBER=+12764458808
PUBLIC_BASE_URL=https://armlike-unadduced-ai.ngrok-free.dev

AZURE_SPEECH_KEY=CGLZa3AA0p0XwnVR...
AZURE_SPEECH_REGION=eastus
```

21. SECURITY FEATURES

Feature	Implementation
Password Storage	PBKDF2-SHA256, 1 million iterations (slow hash = brute-force resistant)
Brute-Force Protection	7 failures in 10 min = 15 min lockout, tracked by username+IP
Session Security	HTTPS-only cookies, HttpOnly (no JS access), SameSite=Lax (CSRF)
Session Lifetime	8 hours maximum
Audit Logging	All login/logout events logged to auth_log.jsonl
Route Protection	/admin/* requires login; Twilio webhooks are public
Atomic File Writes	Temp file + os.replace() prevents data corruption
Phone Masking	Numbers masked in logs: +1*****7890
Admin Token	API access via ADMIN_TOKEN header (alternative to session)

22. KEY DESIGN DECISIONS AND WHY

Decision	Why
File-based storage (no database)	Simplicity, portability, no setup needed. JSON files are easy to inspect and backup.
Daemon threads (not processes)	Simpler than multiprocessing, shares memory, auto-cleanup on exit.
Hybrid speech approach	Twilio Gather for live IVR (real-time), Whisper for post-call (high accuracy).
Text hash caching for TTS	Azure TTS costs money per character. Same text is synthesized only once.
Chunked translation	Google Translate has character limits. 3000-char chunks with retry prevents failures.
Configurable surveys	Change questions.txt to deploy a different survey — no code changes needed.
Atomic writes	Write to temp file then <code>os.replace()</code> . If crash mid-write, original file is intact.
Max 3 attempts	Respects participant's time. Stop calling after 3 unanswered attempts.
1-hour retry gap	Don't spam-call someone who just didn't answer.
-15% TTS prosody	Kiswahili Neural TTS speaks too fast by default. Slowing improves comprehension.
Heartbeat monitoring	Detect crashed background threads before researchers notice missing data.
Inbound support	Participants in low-connectivity areas may prefer to call in on their own schedule.
Direction tracking	Researchers need to distinguish self-initiated from scheduled calls in analysis.

23. LIBRARIES AND WHAT EACH ONE DOES

Library	Version	Purpose
<code>flask</code>	3.1.2	Web framework — HTTP routes, sessions, templates
<code>twilio</code>	9.10.0	Twilio API client — places calls, generates TwiML
<code>openai-whisper</code>	20240930	Speech-to-text AI — transcribes Kiswahili audio
<code>azure-cognitiveservices-speech</code>	1.48.1	Azure Neural TTS — Kiswahili voice prompts
<code>googletrans</code>	4.0.0rc1	Google Translate — Kiswahili to English
<code>gtts</code>	2.5.4	Google TTS — generates English audio from text
<code>deepfilternet</code>	0.5.6	Neural noise removal — cleans call recordings
<code>torch</code>	2.8.0	PyTorch ML framework — powers DeepFilterNet
<code>torchaudio</code>	2.8.0	Audio processing for PyTorch
<code>numpy</code>	1.26.4	Numerical computing — audio data as arrays
<code>pandas</code>	2.3.3	Data manipulation — builds Excel export DataFrames
<code>openpyxl</code>	3.1.5	Excel file generation — writes .xlsx files
<code>pyyaml</code>	6.0.3	YAML parser — reads config.yaml
<code>python-dotenv</code>	1.2.1	Environment variables — loads .env file
<code>werkzeug</code>	(Flask dep)	Password hashing — PBKDF2 for auth
<code>colorlog</code>	6.10.1	Colored console output — prettier logs
<code>requests</code>	(dep)	HTTP client — downloads recordings from Twilio
<code>pydub</code>	0.25.1	Audio manipulation — format conversion

24. GLOSSARY

Term	Meaning
IVR	Interactive Voice Response — automated phone system that interacts with callers
TwiML	Twilio Markup Language — XML format for scripting phone call behavior
DTMF	Dual-Tone Multi-Frequency — the technical name for phone keypad tones
TTS	Text-to-Speech — converting written text into spoken audio
STT	Speech-to-Text — converting spoken audio into written text
SSML	Speech Synthesis Markup Language — XML for controlling TTS voice prosody
Webhook	An HTTP callback — a URL that Twilio calls to notify your app of events
ngrok	A tunnel service that exposes local servers to the internet via HTTPS
PBKDF2	Password-Based Key Derivation Function 2 — slow hash for passwords
RLock	Reentrant Lock — same thread can acquire it multiple times, prevents deadlock
Daemon Thread	Background thread that dies automatically when the main program exits
E.164	International phone number format (e.g., +12764458808)
PCM	Pulse-Code Modulation — raw uncompressed audio format
WAV	Waveform Audio File Format — uncompressed audio container
MP3	Compressed audio format (MPEG Layer 3)
DeepFilterNet	Neural network for real-time speech enhancement and noise removal
Whisper	OpenAI speech recognition model — supports 99 languages including Kiswahili
Flask Blueprint	A way to organize Flask routes into separate modules
Atomic Write	Write to temp file then rename — prevents partial or corrupted writes
Heartbeat	Periodic signal from a thread indicating it is alive and functioning
Inbound Call	Call initiated by the participant dialing the Twilio number
Outbound Call	Call placed by the system to the participant via the Twilio API

25. PRESENTATION TALKING POINTS

Opening

"AudioSurvey AI is a production-grade platform that automates voice-based research surveys in Kiswahili for African refugee populations, powered by AI and cloud telephony."

Problem Statement

"Traditional surveys require in-person enumerators — expensive, hard to scale, and dangerous in conflict-affected areas. Our system automates the entire process over regular phone calls. Any phone, any location, no smartphone required."

Technical Highlights (v1.2.0)

1. "We use **Twilio** to place automated outbound calls and accept inbound calls, so participants can choose when to take the survey"
2. "Questions are spoken using **Azure Neural TTS** with Kiswahili voice **sw-KE-ZuriNeural** at -15% prosody for clarity"
3. "Responses are collected via **speech recognition** (open-ended) and **keypad presses** (multiple choice)"
4. "After the call, recordings go through a **4-stage ML pipeline**: DeepFilterNet noise removal, Whisper large-v3 transcription, Google Translate, and gTTS English audio generation"
5. "The system is protected by **PBKDF2 password hashing**, brute-force lockout, and session security — all in a web-based admin dashboard"
6. "Three background threads run simultaneously: Flask for HTTP, the scheduler for placing calls every 15 seconds, and the ML worker for processing recordings — coordinated via a thread-safe heartbeat monitor"
7. "In v1.2.0 we added **inbound call handling**, **runtime health monitoring** (detect crashed threads), and **admin call controls** (end individual or all calls)"

Architecture in One Sentence

"Flask + 3 threads (scheduler, ML worker, main) share thread-safe JSON state, with heartbeat monitoring and atomic writes protecting data integrity."

Scale and Impact

"The survey contains 76 questions across 4 types. The platform automates the full research pipeline from call placement through Excel data export — zero manual intervention required."

End of Study Guide — AudioSurvey AI v1.2.0 | April 6, 2026